

COS 710 Assignment 2

Structure-Based Genetic Programming for Regression

Name: Diya Budhia
Student Number: u22594044

20 April 2026

Contents

1	Description of the Data Set	2
2	Description of the Performance Metric Used to Assess the Quality of the Predictor	2
3	Description of How Structure is Incorporated into Canonical GP	2
4	Description of the Representation Used and the Terminal and Function Sets	3
5	Description of Initial Population Generation	3
6	Description of the Fitness Function and Fitness Evaluation	3
7	Description of the Selection Method	4
8	Description of Genetic Operators	4
9	Description of the Experimental Setup	4
10	Results and Discussion of the Results	5
11	Runtimes of the Algorithm	7
12	Comparison of the Results with Assignment 1	8
13	Conclusion	9

1 Description of the Data Set

The dataset used in this assignment is `Residential_Energy_Dataset_UK- 2014-2020.csv`. The full dataset contains 201,604 rows and 7 columns. The available attributes are `utc_timestamp`, `Electricity_load`, `Residential_electricity_price`, `Residential_solar_generation`, `Residential_wind_generation`, `Temperature`, and `Relative_Humidity`. The target variable for prediction is `Electricity_load`.

The timestamp spacing is 15 minutes, which implies 96 samples per day. For consistency with Assignment 1 and to keep runtimes manageable for repeated runs, the first 5,000 rows were used in the reported experiments. Time ordering was preserved throughout preprocessing and splitting.

Two forecasting formulations from the assignment brief were evaluated:

- **Previous days at the same time:** predict load using the same time point from the previous n days.
- **Previous values:** predict load using the previous m consecutive values.

For the `previous_days` experiment, $n = 7$, which yields lag offsets:

$$[96, 192, 288, 384, 480, 576, 672]$$

For the `previous_values` experiment, $m = 24$, which yields:

$$[1, 2, \dots, 24]$$

2 Description of the Performance Metric Used to Assess the Quality of the Predictor

The primary fitness metric is root mean squared error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Since this is a minimization problem, lower RMSE indicates better fitness.

Additional reporting metrics:

- Mean absolute error (MAE)
- Mean absolute percentage error (MAPE)
- Hit ratio using bound $\epsilon = 0.01$

A prediction is counted as a hit when:

$$|y_i - \hat{y}_i| \leq \epsilon$$

The hit ratio is the proportion of test cases satisfying this condition.

3 Description of How Structure is Incorporated into Canonical GP

This assignment extends canonical GP with explicit structure-based mechanisms:

- **Global-local search alternation:** evolution alternates between global and local search modes.

- **Structural similarity control:** mate selection and mode switching use a global similarity threshold index T_g .
- **Global area cutoff depth:** during global search, trees are constrained by cutoff depth D_g .
- **Transferred global area initialization:** when switching from global to local search, new individuals are injected near the best region using transfer depth DI_g .
- **Structural diversity pressure:** tournament tie-breaking prefers less common structure signatures.

The practical mode-switch condition is based on a no-change tolerance window W_g and similarity threshold T_g . If no validation improvement is observed for W_g generations and population similarity is high enough, search mode is switched.

4 Description of the Representation Used and the Terminal and Function Sets

The system uses tree-based symbolic regression.

- Internal nodes: functions
- Leaf nodes: terminals

Terminal set:

- Lag variables $x_0, x_1, \dots$
- Ephemeral random constants sampled from $[-5, 5]$

Function set:

- Binary: add, sub, mul, protected division
- Unary: sin, cos, log, exp

Numerical protection is used for division, logarithm, and exponential overflow handling.

5 Description of Initial Population Generation

The initial population is generated with ramped half-and-half:

- **Full:** function nodes until depth limit
- **Grow:** mixed terminal/function expansion before depth limit

Initial depth is controlled by parameter DI .

6 Description of the Fitness Function and Fitness Evaluation

Fitness is computed on the training split and combines prediction error with structural regularization:

$$f(\text{ind}) = \text{RMSE}(y_{\text{train}}, \hat{y}_{\text{train}}) + \lambda \cdot \text{size}(\text{tree}) + \alpha \cdot \text{depthPenalty}(\text{tree})$$

where:

- λ is the parsimony coefficient
- α is the structural depth penalty coefficient

Data was split in temporal order as:

- 80% training
- 10% validation
- 10% test

Validation RMSE is used to track the best individual across generations, and final metrics are reported on the test set.

7 Description of the Selection Method

Tournament selection is used with tournament size S . In case of similar fitness, preference is given to structurally rarer individuals (based on tree structure signatures), then to simpler trees. This supports diversity and prevents premature structural convergence.

8 Description of Genetic Operators

Subtree Crossover

Random crossover points are selected in two parents and subtrees are swapped. Structural compatibility is encouraged by preferring same-arity swap points. Offspring violating depth limits are rejected in favor of valid parents.

Mutation

A combination of subtree mutation and point mutation is used. Mutation subtree size is constrained by parameter D_u (maximum mutation subtree depth). If trees are too deep, mutation tends to shrink structures.

Mode-aware Search

- **Global search** runs for S_g generations (or until stagnation trigger).
- **Local search** uses tolerance window W_g and higher exploitation pressure.
- Switches occur when optimum/stagnation is reached, and transferred-area initialization with depth DI_g is applied when entering local mode.

9 Description of the Experimental Setup

Language used: Python.

Machine specifications:

- Operating system: Windows 10
- Architecture: AMD64
- Processor: Intel64 Family 6 Model 170 Stepping 4, GenuineIntel
- Python version: 3.11.8

- Logical CPU count: 14

Main SBGP parameters:

- $P = 120$ (population size)
- $R = (0.85, 0.15)$ (crossover, mutation rates)
- $S = 4$ (tournament size)
- $D_m = 9$ (max tree depth)
- $DI = 4$ (initial tree depth)
- $D_u = 4$ (mutation max subtree depth)
- $S_g = 30$ (global-search generations)
- $D_g = 9$ (global area cutoff depth)
- $T_g = 0.75$ (global similarity threshold index)
- $W_g = 10$ (no-change tolerance window)
- $DI_g = 3$ (initial depth in transferred global area)
- Number of runs: 10 per mode
- Hit bound: $\epsilon = 0.01$
- Base seed: 4, with seed per run = base seed + run index

Two modes were tested:

1. `previous_days` with $n = 7$
2. `previous_values` with $m = 24$

10 Results and Discussion of the Results

Table 1 summarizes 10-run SBGP results for both modes.

Table 1: Assignment 2 SBGP summary over 10 runs

Mode	Mean RMSE	Std. RMSE	Best RMSE	Mean Hit Ratio
Previous 7 days	0.013686	0.000218	0.013359	49.58%
Previous 24 values	0.012687	0.000330	0.012498	54.10%

Additional SBGP metrics:

Table 2: Additional metrics over 10 runs (mean $\pm$ std)

Mode	Mean MAE $\pm$ Std	Mean MAPE $\pm$ Std	Best Hit Ratio
Previous 7 days	0.011256 $\pm$ 0.000196	12.716% $\pm$ 0.237%	51.27%
Previous 24 values	0.010345 $\pm$ 0.000395	11.667% $\pm$ 0.530%	55.02%

Best expressions:

- **Previous 7 days** (best run index 6): $(x_6 / \exp((x_6 - x_4)))$
- **Previous 24 values** (best run index 1): $(x_0 * \cos((\log(\exp(x_0)) + \exp(\log(x_0)))))$

Run-level Results (Seeds and RMSE)

Table 3: Seeds and test RMSE for previous 7 days SBGP (10 runs)

Run	Seed	Test RMSE	Hit Ratio
1	4	0.013625	49.19%
2	5	0.013906	49.65%
3	6	0.013790	49.19%
4	7	0.013368	51.27%
5	8	0.014039	51.27%
6	9	0.013359	48.96%
7	10	0.013679	49.19%
8	11	0.013906	49.65%
9	12	0.013679	49.19%
10	13	0.013503	48.27%

Table 4: Seeds and test RMSE for previous 24 values SBGP (10 runs)

Run	Seed	Test RMSE	Hit Ratio
1	4	0.012498	54.22%
2	5	0.013672	47.79%
3	6	0.012585	55.02%
4	7	0.012567	54.82%
5	8	0.012584	55.02%
6	9	0.012585	55.02%
7	10	0.012624	54.02%
8	11	0.012584	55.02%
9	12	0.012585	55.02%
10	13	0.012585	55.02%

Discussion

The `previous_values` configuration outperformed `previous_days` across every reported metric: lower mean RMSE (0.012687 vs 0.013686), lower mean MAE (0.010345 vs 0.011256), lower mean MAPE (11.67% vs 12.72%), and higher mean hit ratio (54.10% vs 49.58%). This suggests that short-lag consecutive values carry more predictive signal for 15-minute electricity load than same-time-of-day values from the previous week.

Variance across runs was low for both modes (RMSE std of 0.000218 and 0.000330 respectively), confirming that the SBGP algorithm is stable and not sensitive to random seed. The `previous_values` mode did exhibit slightly higher variance, likely because the 24-dimensional input space allows more structural diversity in evolved trees.

The structural dynamics plot (Figure 2) illustrates a key property of the SBGP approach: population variety decreases steadily across generations while structural similarity rises, reflecting the convergence pressure exerted by the global-local search alternation and the T_g threshold. The mode-switching mechanism ensured that the search remained broad in early generations (global phase, D_g -constrained trees) and shifted to exploitation after stagnation was detected.

The best evolved expressions are compact and interpretable. The `previous_days` best expression (`x6 / exp((x6 - x4))`) involves only two lag inputs and an exponential decay

relationship, while the `previous_values` best expression uses only `x0` with nested trigonometric and logarithmic transformations, suggesting the most recent value carries substantial load information at 15-minute resolution.

The convergence curves (Figure 1) show rapid initial improvement in the first 10–15 generations, followed by oscillation consistent with the alternating global-local phases. Neither mode appears to fully plateau by generation 60, indicating further improvement may be achievable with more generations.

Graphs

Figures 1, 2, and 3 provide visual summaries from the generated experiment CSV files.

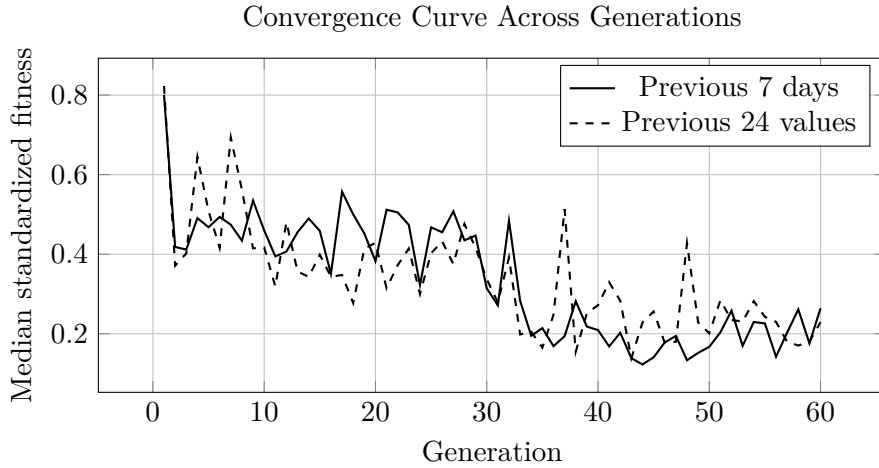


Figure 1: Median standardized fitness per generation (aggregated over 10 runs per mode). Lower values indicate better fitness.

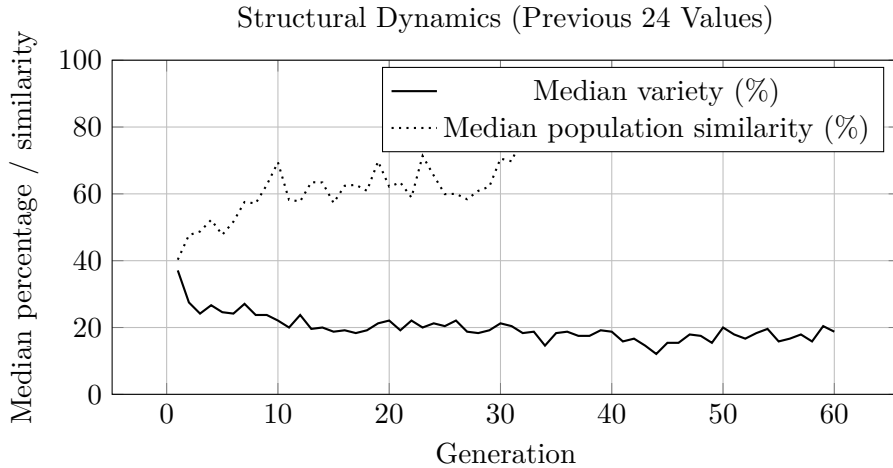


Figure 2: Structural dynamics across generations for the previous-values mode: diversity decreases while structural similarity increases, reflecting convergence under structure-based control.

11 Runtimes of the Algorithm

Runtime summary over 10 runs:

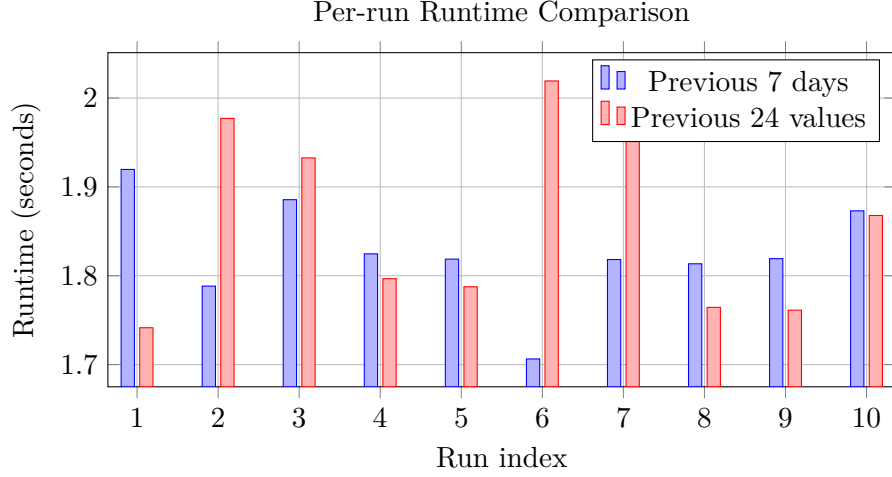


Figure 3: Runtime per run for both modes (10 runs each).

- **Previous 7 days:** mean runtime = 1.8267 s, std = 0.0553 s, total runtime = 18.2673 s
- **Previous 24 values:** mean runtime = 1.8668 s, std = 0.1057 s, total runtime = 18.6681 s

These runtimes are for the 5,000-row subset and the parameter settings listed in Section 9.

12 Comparison of the Results with Assignment 1

Table 5 compares Assignment 2 SBGP with the Assignment 1 canonical GP results from the Assignment 1 report.

Table 5: Assignment 1 vs Assignment 2 comparison (10-run means)

Setting	A1 Mean RMSE	A2 Mean RMSE	A1 Mean Hit	A2 Mean Hit
Previous 7 days	0.013171	0.013686	50.28%	49.58%
Previous 24 values	0.013055	0.012687	51.49%	54.10%

Runtime comparison:

- **Previous 7 days:** A1 mean 2.8254 s vs A2 mean 1.8267 s
- **Previous 24 values:** A1 mean 2.1180 s vs A2 mean 1.8668 s

Interpretation:

- For `previous_values`, SBGP improved both accuracy (lower RMSE) and hit ratio compared to Assignment 1.
- For `previous_days`, Assignment 1 had slightly better mean RMSE and hit ratio in this particular setup.
- SBGP achieved consistently lower runtime in both modes, likely due to stronger structural constraints and faster convergence behavior.

13 Conclusion

This assignment implemented a complete structure-based GP system for electricity load forecasting and evaluated it over the required 10 runs for each lag formulation. The report covered all required specification items: dataset, metrics, structure integration, representation, initialization, fitness, selection, operators, setup, multi-run statistics, runtime, and comparison with Assignment 1.

Overall, SBGP provided strong results, particularly for the `previous_values` formulation, where it achieved lower mean RMSE and higher hit ratio than Assignment 1 while maintaining fast execution. The global-local structural search schedule with parameters S_g , D_g , T_g , W_g , and DI_g provided a clear and effective mechanism to incorporate structure directly into the evolutionary process.